Introduction to DevOps

DevOps

Devops is a set of practices, tools, and a cultural philosophy that automate and integrate the processes between software development and it teams. it emphasizes team empowerment, cross-team communication and collaboration, and technology automation.

A devops team includes developers and IT operations working collaboratively throughout the product lifecycle, in order to increase the speed and quality of software deployment. it’s a new way of working, a cultural shift, that has significant implications for teams and the organizations they work for.

DevOps stakeholder

In a DevOps environment, stakeholders are individuals or groups who have an interest or concern in the development and operation of a software system. Their involvement and collaboration are key to the success of DevOps initiatives. They are the primary stakeholders in a typical DevOps setup.

In a successful DevOps culture, these stakeholders work collaboratively, breaking down traditional silos to improve efficiency, enhance quality, and rapidly deliver value to customers. Effective communication, shared goals, and mutual respect among all stakeholders are key to realizing the full benefits of DevOps.

Development Team:

- **Role**: Responsible for writing, testing, and maintaining the software. They implement new features and fix bugs.
- **Interest**: In a DevOps culture, developers are interested in streamlined processes, tools that aid in continuous integration and continuous deployment, and a close collaboration with operations to ensure smooth software delivery.

Operations Team:

- **Role**: Focuses on the deployment, availability, and reliability of the software.
- **Interest**: They aim for stable and efficient operational environments and tools that assist in monitoring, automation, and quick resolution of operational issues.

Quality Assurance (QA) Team:

- **Role**: Ensures the quality of the software through testing and validation.
- **Interest**: QA stakeholders seek integration into the CI/CD pipeline for automated testing and early detection of issues.

Security Team (DevSecOps):

- **Role**: Integrates security at every phase of the software development life cycle.
- **Interest**: Their focus is on implementing security measures and compliance standards seamlessly within the DevOps workflow.

Product Owners/Managers:

- **Role**: Define the product vision and prioritize the development backlog.
- **Interest**: Interested in aligning software development with business objectives and ensuring that the final product meets customer needs.

Customers and End-Users:

- **Role**: The ultimate recipients of the software.
- **Interest**: They are interested in reliable, functional, and user-friendly software. Continuous feedback from customers is crucial for iterative improvement.

IT Leadership and Management:

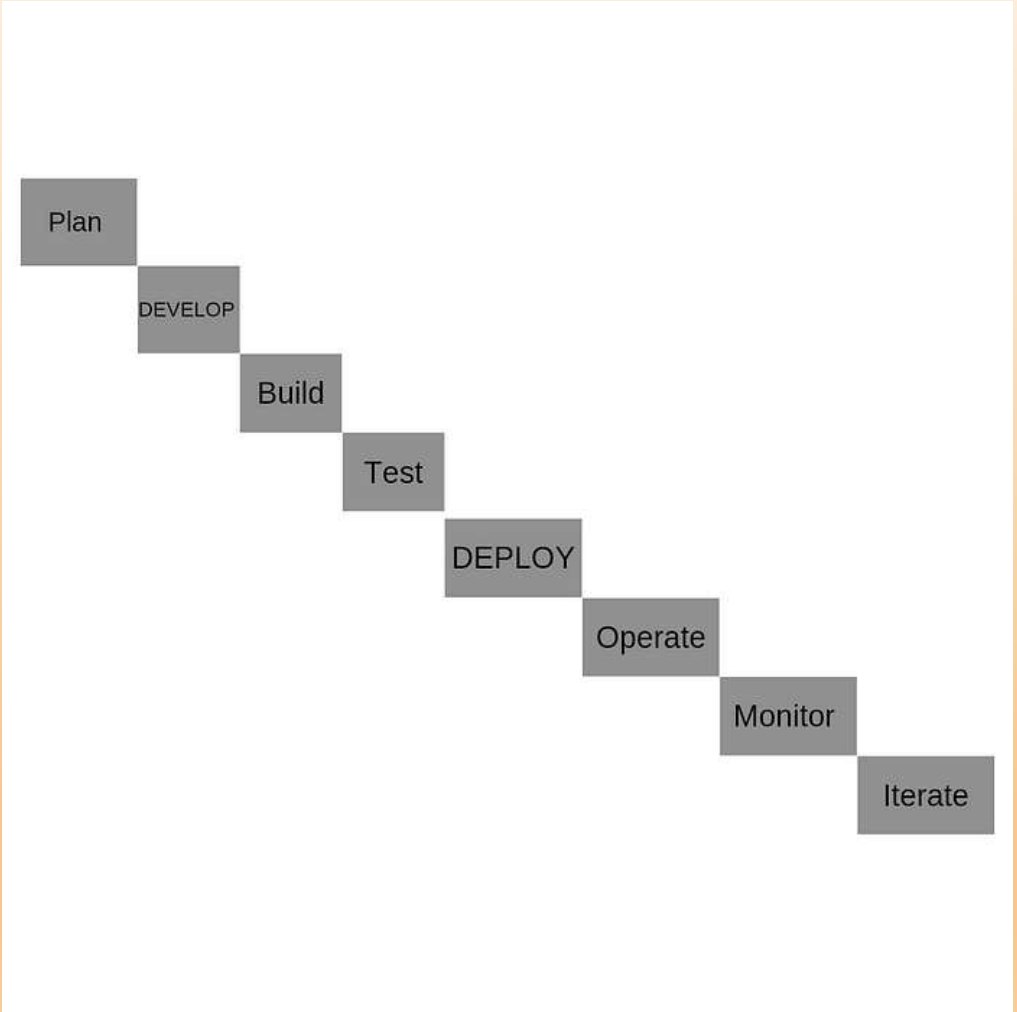
- **Role**: Provide strategic direction, resources, and support for the DevOps initiatives.
- **Interest**: They focus on the overall efficiency, cost-effectiveness, and alignment of DevOps practices with the organization’s goals.

External Partners and Vendors:

- **Role**: Supply necessary tools, technologies, or services.
- **Interest**: They are interested in maintaining a symbiotic relationship where their tools and services effectively support the organization’s DevOps objectives.

SDLC Models, Lean, ITIL, Agile

SDLC Model



Lean

Lean DevOps is about delivering the outcomes that matter. It combines the principles of Lean with the Continuous Delivery and service ownership of DevOps

Lean is a structured process that organizes tasks and allows oversight. Instead of 5 common stages, Lean has 5 core principles.

Define a value: Which activities are time wasters? Which ones add value to the project? That's the distinction you must make here. Consider your customers. Does your effort provide value to them, whether directly or indirectly?

Map the value stream: The second principle requires a visualization of your customer value activities. This lets you keep the project on task and moving forward, especially if you use an Agile project management style, such as scrum. You can use a Kanban board, such as the one offered in Jira Software, for this.

Create a flow: You want your project to flow seamlessly. Any blockage can be detrimental. Keep an eye out for any potential clots. If one occurs, analyze what caused it and how to avoid it. For example, clots tend to form while awaiting stakeholder feedback. Prevent this by limiting the chunks of work for review.

Establish pull: Shoving new work on your team when they're at capacity can hinder flow. Start new work only when there's demand and your team has time. A pull system creates a work queue, meaning a team member without current work can pull the first high-priority ticket to focus on.

Seek perfection: Continuous improvement is the foundation of Lean project management. You and your team should be better than you were yesterday. Analyze performance and identify opportunities for improvement. Remember, you want to ensure you're providing customer value. If something isn't working, examine why and make incremental changes based on that.

DevOps vs. Lean principles

Customer orientation: Both methods give importance to the customer. In Lean, you choose the customer value activities that matter. DevOps creates customer empathy image mapping, which breaks down business goals into something meaningful for the client.

Focus: Lean principles seek to optimize across the entire project. DevOps wants to integrate development and operations through cross-collaboration and documentation.

Execution vs. vision: Lean is all about improving execution for a better result, but DevOps has loftier goals. It leverages cross-functional teams and automation to effect systemic changes in a company.

Automation: DevOps is all about automation. Lean isn't. With automation, DevOps employs tech to check and deploy code, run tests, and pull requests. That way, someone on your team won’t need to do this manually.

Timelines: Lean timelines center around sprints, which could stretch into months. DevOps sometimes requires delivery on an hourly basis.

Agile

Agile is an iterative approach to project management and software development that focuses on collaboration, customer feedback, and rapid releases. It arose in the early 2000s from the software development industry, helping development teams react and adapt to changing market conditions and customer demands.

Agile emphasizes collaboration between developers and product management — DevOps includes the operations team

Agile centers the flow of software from ideation to code completion — DevOps extends the focus to delivery and maintenance

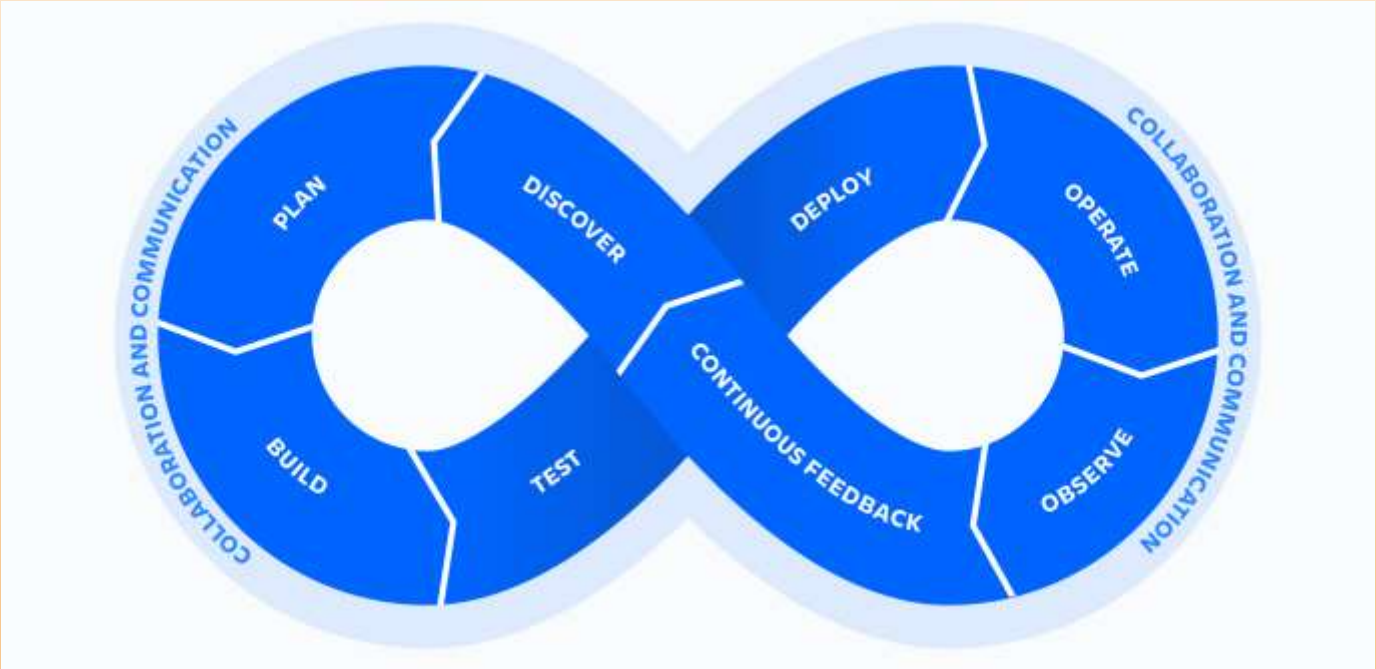
Agile emphasizes iterative development and small batches — DevOps focuses more on test and delivery automation

Agile adds structure to planned work for developers — DevOps incorporates unplanned work common to operations teams

ITIL

ITIL (Information Technology Infrastructure Library) is a set of guidelines for IT service management. The guidelines cover best practices and tried-and-true processes for everything from incident management to problem management to change management.

Lifecycle of DevOps



Benefits of DevOps

Speed

Rapid Delivery

Reliability

Scale

Improved collaboration

Security

Configuration management in DevOps

Configuration management occurs when a configuration platform is used to automate, monitor, design and manage otherwise manual configuration processes. System-wide changes take place across servers and networks, storage, applications, and other managed systems.

A number of tools are available for those seeking to implement configuration management in their organizations. Puppet has carried the torch in pioneering configuration management, but other companies like Chef and Red Hat also offer intriguing suites of products to enhance configuration management processes. Proper configuration management is at the core of continuous testing and delivery, two key benefits of DevOps.

Components: Configuration Management in DevOps

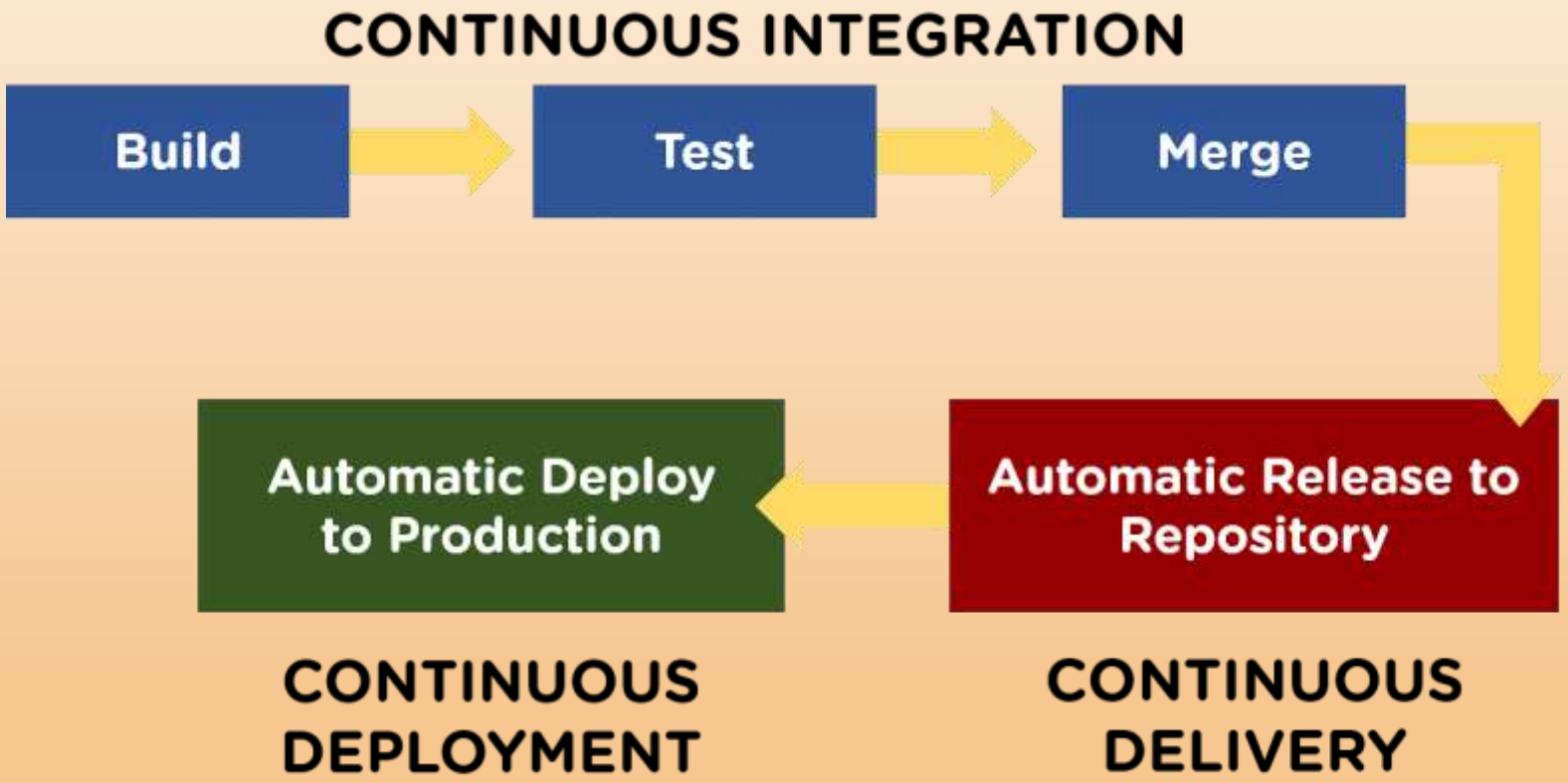
Configuration management takes on the primary responsibility for three broad categories required for DevOps transformation: identification, control, and audit processes.

Identification:
The process of finding and cataloging system-wide configuration needs.

Control:
During configuration control, we see the importance of change management at work. It’s highly likely that configuration needs will change over time, and configuration control allows this to happen in a controlled way as to not destabilize integrations and existing infrastructure.

Audit:
Like most audit processes, a configuration audit is a review of the existing systems to ensure that it stands up to compliance regulation and validations.

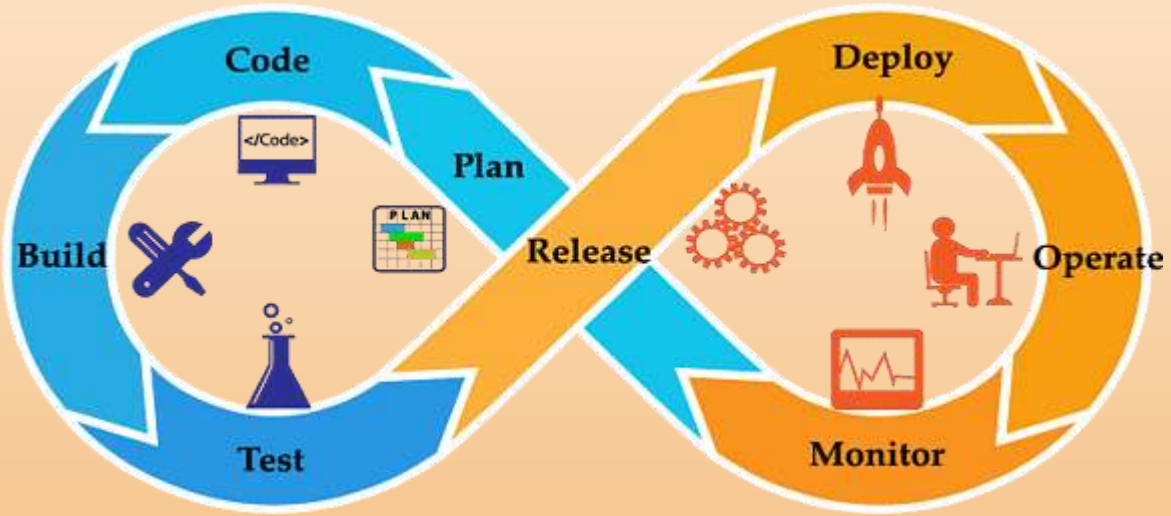
Continuous integration and Deployment in Devops



What is Continuous Integration?

Continuous Integration (CI) is a DevOps software development practice that enables the developers to merge their code changes in the central repository. That way, automated builds and tests can be run. The amendments by the developers are validated by creating a built and running an automated test against them.

In the case of Continuous Integration, a tremendous amount of emphasis is placed on testing automation to check on the application. This is to know if it is broken whenever new commits are integrated into the main branch


Code
Plan
Build
Release
Operate
Deploy
Test
Monitor
</Code>
PLAN

What is Continuous Delivery?

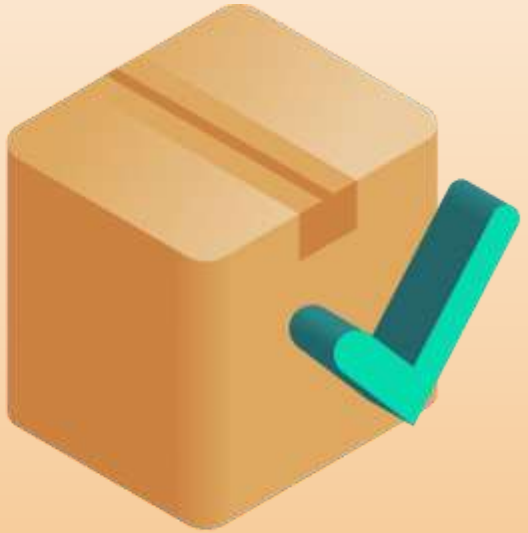
Continuous Delivery (CD) is a DevOps practice that refers to the building, testing, and delivering improvements to the software code. The phase is referred to as the extension of the Continuous Integration phase to make sure that new changes can be released to the customers quickly in a substantial manner.

This can be simplified as, though you have automated testing, the release process is also automated, and any deployment can occur at any time with just one click of a button.

Continuous Delivery gives you the power to decide whether to make the releases daily, weekly, or whenever the business requires it. The maximum benefits of Continuous Delivery can only be yielded if they release small batches, which are easy to troubleshoot if any glitch occurs.

Building

Testing

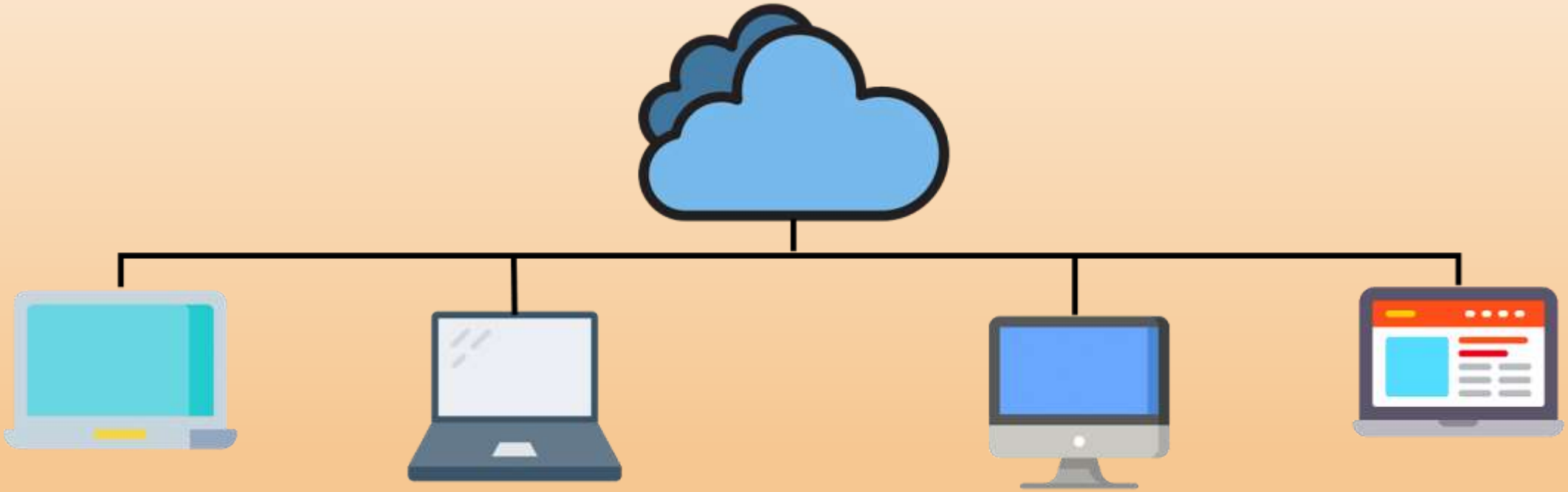


Delivery

What is Continuous Deployment?

When the step of Continuous Delivery is extended, it results in the phase of Continuous Deployment. Continuous Deployment (CD) is the final stage in the pipeline that refers to the automatic releasing of any developer changes from the repository to the production.

Continuous Deployment ensures that any change that passes through the stages of production is released to the end-users. There is absolutely no way other than any failure in the test that may stop the deployment of new changes to the output. This step is a great way to speed up the feedback loop with customers and is free from human intervention.



Linux OS Introduction

The Linux Operating System is a type of operating system that is similar to Unix, and it is built upon the Linux Kernel. The Linux Kernel is like the brain of the operating system because it manages how the computer interacts with its hardware and resources. It makes sure everything works smoothly and efficiently. But the Linux Kernel alone is not enough to make a complete operating system. To create a full and functional system, the Linux Kernel is combined with a collection of software packages and utilities, which are together called Linux distributions. These distributions make the Linux Operating System ready for users to run their applications and perform tasks on their computers securely and effectively. Linux distributions come in different flavors, each tailored to suit the specific needs and preferences of users.

Linux Distribution

Linux distribution is an operating system that is made up of a collection of software based on Linux kernel or you can say distribution contains the Linux kernel and supporting libraries and software. And you can get Linux based operating system by downloading one of the Linux distributions and these distributions are available for different types of devices like embedded devices, personal computers, etc. Around **600 + Linux Distributions** are available and some of the popular Linux distributions are:

MX Linux

Manjaro

Linux Mint

elementary

Ubuntu

Debian

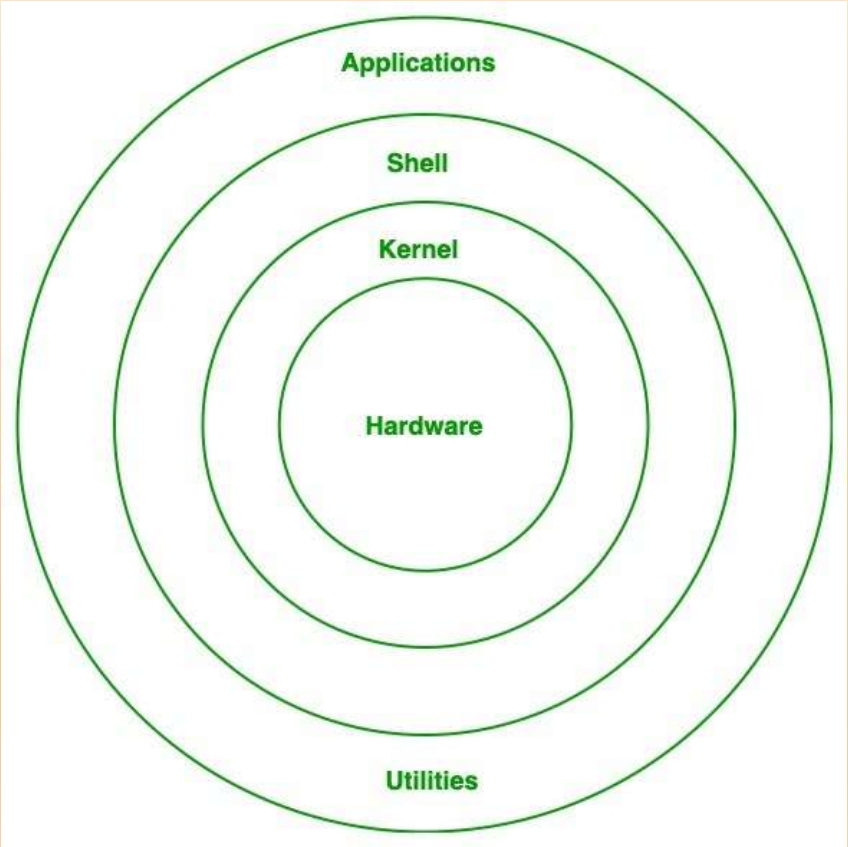
Solus

Fedora

openSUSE

Deepin

Architecture of Linux



Importance of Linux OS in DevOps

In the world of DevOps, where automation and collaboration are the keys to success, Linux stands as a cornerstone. Linux, an open-source operating system, plays a pivotal role in the DevOps ecosystem, enabling development and operations teams to work together seamlessly, streamline processes, and deliver high-quality software at an accelerated pace.

Stability and Reliability. In DevOps, system stability and reliability are paramount. Linux is renowned for its stability and uptime, making it an ideal choice for hosting critical applications and services. The predictable performance of Linux helps DevOps teams minimize downtime and maintain a robust infrastructure.

Installation of Linux

Hardware Requirements

- 64-bit processor, x86-64, 2 GHz or faster
- At least 2 GB free hard drive space to install DataConnect
- 8 GB high-speed RAM

Runtime Engine Requirements

In addition to the minimum requirements for your operating system, each additional running Runtime Engine on the same machine requires the following:

- 2 GB high speed RAM
- 1–2 available processor cores
- A valid engine license

Linux basic commands utilities

Pwd

Mkdir

Ls

Rmdir

Cd

Touch

Cat

Rm

Cp

useradd

passwd

groupadd

id

rename

mv

Environment variables in linux

Environment variables, often referred to as ENVs, are dynamic values that wield significant influence over the behavior of programs and processes in the Linux operating system. These variables serve as a means to convey essential information to software and shape how they interact with the environment. Every Linux process is associated with a set of environment variables, which guide its behavior and interactions with other processes.

Accessing Environment Variables

In Linux, the primary conduit for interacting with environment variables is the shell. The shell acts as a command-line interpreter, executing instructions entered by the user. The most prevalent shell in the Linux world is the Bash shell (Bourne Again SHell), which comes as the default in many Linux distributions.

Scope of an environment variable

Understanding the scope of an environment variable is crucial. It dictates where the variable can be accessed or defined, making a clear distinction between global and local scopes.

RPM and Yum

RPM is a popular package management tool in Red Hat Enterprise Linux-based distros. Using RPM, you can install, uninstall, and query individual software packages. Still, it cannot manage dependency resolution like YUM. RPM does provide you useful output, including a list of required packages. An RPM package consists of an archive of files and metadata. Metadata includes helper scripts, file attributes, and information about packages.

What is RPM

RPM is a command-line package manager developed in 1995 by Red Hat. The package manager was designed to work on Red Hat-based systems. Today, RPM is the core component of many Linux distributions, including CentOS, Fedora, Oracle Linux, openSUSE, Mageia, etc.

The RPM package manager allows users to query, verify, install, upgrade, and remove packages. **The main downside** is that it doesn't resolve package dependencies or automatic package updates.

What is YUM

YUM (**Y**ellow Dog **U**pdater, **M**odified) is an open-source Linux package management application that uses the RPM package manager. This front-end RPM tool allows users to search official and third-party repositories and install, update, or remove packages from the system.

YUM works with online repositories listed in the */etc/yum.repos.d/*.repo* file. Additionally, the tool allows users to add their own **.repo* files.

YUM's benefits over RPM are automatic updates, easy package management and dependency management.